

WhatNext Vision Motors

Shaping the Future of Mobility with Salesforce CRM

Automated vehicle ordering, dealer assignment, stock control and test-drive engagement

Program	Salesforce Developer (NM), delivered through Skill Wallet
Platform	Salesforce Developer Edition (Lightning Experience)

Project Brief

WhatNext Vision Motors is a vehicle manufacturer and retailer that wants to give customers a smoother buying experience while removing the manual effort behind order handling. This project delivers that goal on the Salesforce platform. A customer places an order, the system finds a dealer that matches the customer's location, refuses the order when the chosen vehicle has no stock, reminds the customer about an upcoming test drive by email, and works through pending orders on a schedule so that their status always reflects the real stock position.

The brief is classified on Skill Wallet as an **Epic** of **medium complexity** with an estimated duration of 45 minutes for the guided build. Figure 1 shows the project workspace and the stages tracked for it.

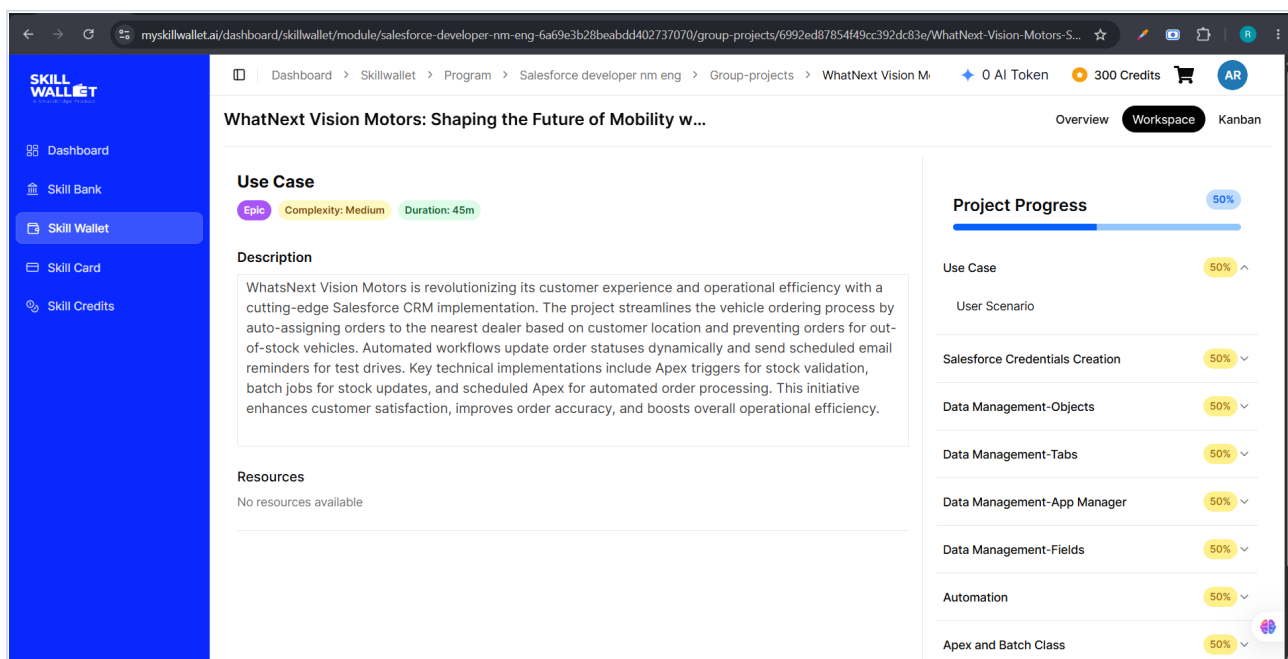


Figure 1: Project workspace on Skill Wallet showing the use case description and the project progress stages

At a glance

Item	Delivered in this project
Custom objects	Six: Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Vehicle Service Request
User interface	Custom tabs for every object and a Lightning app named WhatNext Vision Motors
Declarative automation	Two record-triggered flows: <i>Auto Assign Dealer</i> and <i>Test Drive Reminder</i>
Programmatic automation	Apex trigger with handler class for stock validation, Batch Apex for pending orders, Schedulable Apex for daily execution
Scheduled job	<i>Daily Vehicle Order Processing</i> , running every day at 12:00 PM

Table of Contents

Project Brief	2
1. Introduction	4
1.1 Project Overview	4
1.2 Purpose	4
2. Ideation Phase	5
2.1 Problem Statement	5
2.2 Empathy Map	5
2.3 Brainstorming and Idea Prioritization	6
3. Requirement Analysis	7
3.1 Customer Journey Map	7
3.2 Solution Requirements	7
3.3 Data Flow Diagram	8
3.4 Technology Stack	8
4. Project Design Phase	10
4.1 Problem–Solution Fit	10
4.2 Proposed Solution	10
4.3 Solution Architecture	10
5. Project Planning and Scheduling	12
5.1 Project Planning	12
5.2 Product Backlog and Sprint Schedule	12
5.3 Sprint Tracker, Velocity and Burndown	13
6. Project Development Phase	14
Milestone 1: Salesforce Account	14
Milestone 2: Data Management – Objects	14
Milestone 3: Data Management – Tabs	15
Milestone 4: Data Management – App Manager	16
Milestone 5: Data Management – Fields	17
Milestone 6: Automation	18
Milestone 7: Apex and Batch Class	20
7. Functional and Performance Testing	24
7.1 Functional test cases	24
7.2 Performance observations	25
8. Advantages and Disadvantages	25
9. Conclusion	25
10. Future Scope	25
11. Appendix	26
A. Project artefacts	26
B. Glossary	26

1. Introduction

1.1 Project Overview

Vehicle sales involve several moving parts: a customer enquires, picks a model, places an order, a dealer must be allocated, stock has to be checked, and the customer expects updates along the way. At WhatNext Vision Motors these steps used to depend on registers, spreadsheets and phone calls, which meant that stock figures were often out of date and orders could be accepted for vehicles that were not available.

The Salesforce CRM solution described here brings all of that into one connected system. Vehicles, dealers, customers, orders, test drives and service requests are stored as related custom objects. Flows handle the routine, record-level automation; Apex takes care of rules that need code, such as checking stock inside a database transaction and processing many orders in bulk. The result is a single, trackable workflow from the first order to the final status update.

1.2 Purpose

The purpose of the project is to:

- replace manual vehicle-order handling with an automated, auditable Salesforce workflow;
- stop orders from being created for vehicles that are out of stock, so customers are never promised what cannot be delivered;
- match each order with a dealer based on the customer's location without staff intervention;
- keep customers informed with automatic test-drive reminders and keep order statuses accurate through scheduled batch processing;
- give managers a single place (Lightning app, reports and dashboards) to see how orders and dealers are performing.

2. Ideation Phase

2.1 Problem Statement

The table captures the problem from the point of view of the people who experience it, using the customer problem statement format (I am / I'm trying to / but / because / which makes me feel).

No.	I am	I'm trying to	But	Because	Which makes me feel
PS-1	An operations manager	keep track of vehicle orders and stock	stock details are updated by hand	our systems are not connected to each other	stressed and uncertain
PS-2	A customer	order the vehicle I want	I cannot tell whether it is available	updates reach me late or not at all	anxious and ignored
PS-3	A dealer executive	handle orders for my area	orders reach me without clear allocation	there is no rule that routes an order to a dealer	confused and overworked

2.2 Empathy Map

Two personas sit at the centre of the design: the **Operations Manager**, who owns order accuracy, and the **Customer**, who wants a confident purchase.

Persona 1: Operations Manager

Thinks & feels	Hears	Sees
Worried about wrong orders, stock mismatches and unhappy dealers.	Complaints from dealers and customers about delays and cancellations.	Paper registers, scattered spreadsheets and long email chains.
Says & does	Pains	Gains
Chases people for updates and re-checks stock by hand.	Delays, cancellations and repeated data entry.	Automated checks, one source of truth and live dashboards.

Persona 2: Customer

Thinks & feels	Hears	Sees
Wants to be sure the vehicle is available before committing.	Friends and reviews talking about slow delivery.	Unclear order status and no reminder about the test drive.
Says & does	Pains	Gains
Calls the dealer repeatedly to ask where the order stands.	Waiting, uncertainty and missed appointments.	Quick confirmation, a nearby dealer and timely reminders.

2.3 Brainstorming and Idea Prioritization

Four approaches were considered. Each was scored from 1 (weak) to 5 (strong) against the qualities the business cares about most.

Option	Automation	Scalability	Real-time visibility	Effort to adopt	Total /20
Paper registers	1	1	1	5	8
Spreadsheet tracking	2	2	2	4	10
Stand-alone web portal	4	3	3	2	12
Salesforce CRM	5	5	5	4	19

Salesforce CRM was selected because it combines point-and-click automation with code when needed, grows from one dealer to many, and shows current data to everyone the moment it changes.

Prioritised feature list

Priority	Features
Must have	Vehicle, dealer, customer and order records; stock validation; dealer assignment
Should have	Test-drive reminder email; scheduled batch update of pending orders
Could have	Reports and dashboards for orders and dealer performance
Later	Mobile ordering app, SMS alerts, GPS-based dealer lookup, demand forecasting

3. Requirement Analysis

3.1 Customer Journey Map

The journey runs from the first enquiry to management reporting. The first three stages are handled by the customer; from dealer matching onwards the platform does the work.

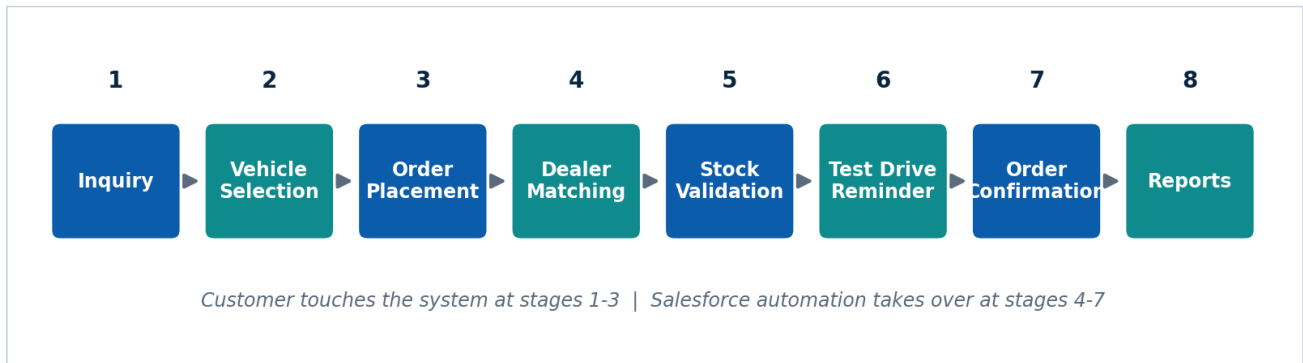


Figure 2: End-to-end customer journey through the order process

Stage	What the customer does	What the system does
1. Inquiry	Contacts the company about a vehicle	Customer record is created in Vehicle Customer
2. Vehicle selection	Chooses a model and variant	Vehicle record shows model, price and stock
3. Order placement	Places the order	Vehicle Order record is saved
4. Dealer matching	-	<i>Auto Assign Dealer</i> flow finds the dealer for the customer's location
5. Stock validation	-	Apex trigger blocks the order when stock is zero
6. Test drive	Books a test drive	<i>Test Drive Reminder</i> flow emails the customer one day before
7. Confirmation	Receives the outcome	Batch job moves pending orders to Confirmed when stock exists
8. Reporting	-	Managers review orders and dealers in reports and dashboards

3.2 Solution Requirements

Functional requirements

ID	Requirement	How it is met
FR-1	Vehicle management	Vehicle object with model, price, stock quantity, status and dealer lookup
FR-2	Dealer management	Vehicle Dealer object with name, location, code, phone and email
FR-3	Customer order creation	Vehicle Order object linked to the customer and the vehicle
FR-4	Stock validation	Before-save Apex trigger rejects orders for vehicles with no stock
FR-5	Dealer auto-assignment	Record-triggered flow looks up the dealer for the customer's location
FR-6	Test-drive scheduling and reminders	Vehicle Test Drive object plus a scheduled-path flow that sends an email
FR-7	Reports and dashboards	Reports and Dashboards tabs available in the Lightning app

Non-functional requirements

Quality	Expectation
Usability	Lightning Experience with one app, clear tabs and list views
Security	Access controlled through profiles; the app is assigned to the System Administrator profile
Reliability	Rules enforced on the server, so every order passes the same checks
Performance	Bulk-safe Apex (one query per transaction) and batch size of 50 for scheduled work
Availability	Cloud hosted, accessible at any time from a browser
Scalability	New dealers, vehicles and locations are just new records

3.3 Data Flow Diagram

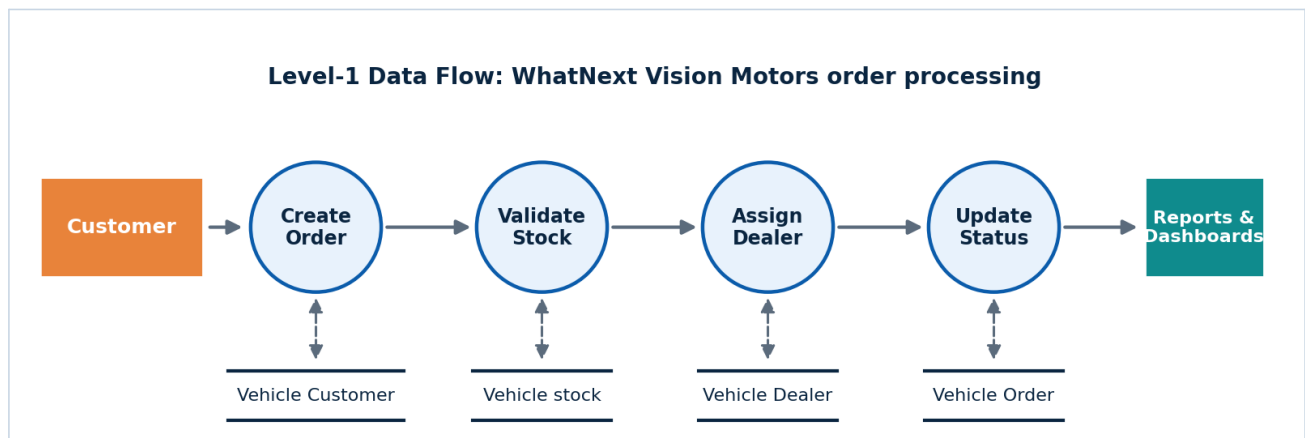


Figure 3: Level-1 data flow for order processing

Order details entered by the customer create a Vehicle Order. The stock check reads the Vehicle record, the dealer step reads the Vehicle Dealer data, and the status update writes the result back to the order. Reports and dashboards read from the same records, so no data is copied between systems.

3.4 Technology Stack

Layer	Technology used
User interface	Salesforce Lightning Experience, custom tabs, Lightning app
Declarative logic	Record-Triggered Flows (including a scheduled path)
Data	Custom objects, fields, picklists and lookup relationships
Server-side rules	Apex trigger with a handler class
Bulk and scheduled processing	Batch Apex and Schedulable Apex
Analytics	Reports and dashboards
Tools	Salesforce Setup, Flow Builder, Developer Console

4. Project Design Phase

4.1 Problem-Solution Fit

Problem observed	How the solution answers it
Stock figures are maintained by hand and go stale	Stock lives on the Vehicle record and is checked by Apex at the moment an order is saved
Customers can order vehicles that are unavailable	The trigger raises a clear error and the order is not saved
Orders are not routed to a dealer consistently	A flow finds the dealer for the customer's location on every new order
Customers forget their test drives	A scheduled path emails them one day ahead
Pending orders are never revisited	A batch job runs daily and confirms orders once stock is available

4.2 Proposed Solution

Parameter	Description
Problem	Manual, disconnected vehicle ordering and stock tracking
Solution	A Salesforce CRM application with automated validation, dealer assignment and scheduled processing
Innovation	Combines flows for quick configuration with Apex for transaction-safe stock rules
Impact	Fewer order errors, faster responses and clearer communication with customers
Business model	Supports the existing dealer network by directing each order to a dealer
Scalability	Handles more dealers, locations and vehicle models without redesign

4.3 Solution Architecture

The architecture has four layers. The data layer holds the six related objects, the automation layer reacts to record changes, the Apex layer enforces rules that need code and processes records in bulk, and the presentation layer exposes everything through one Lightning app.

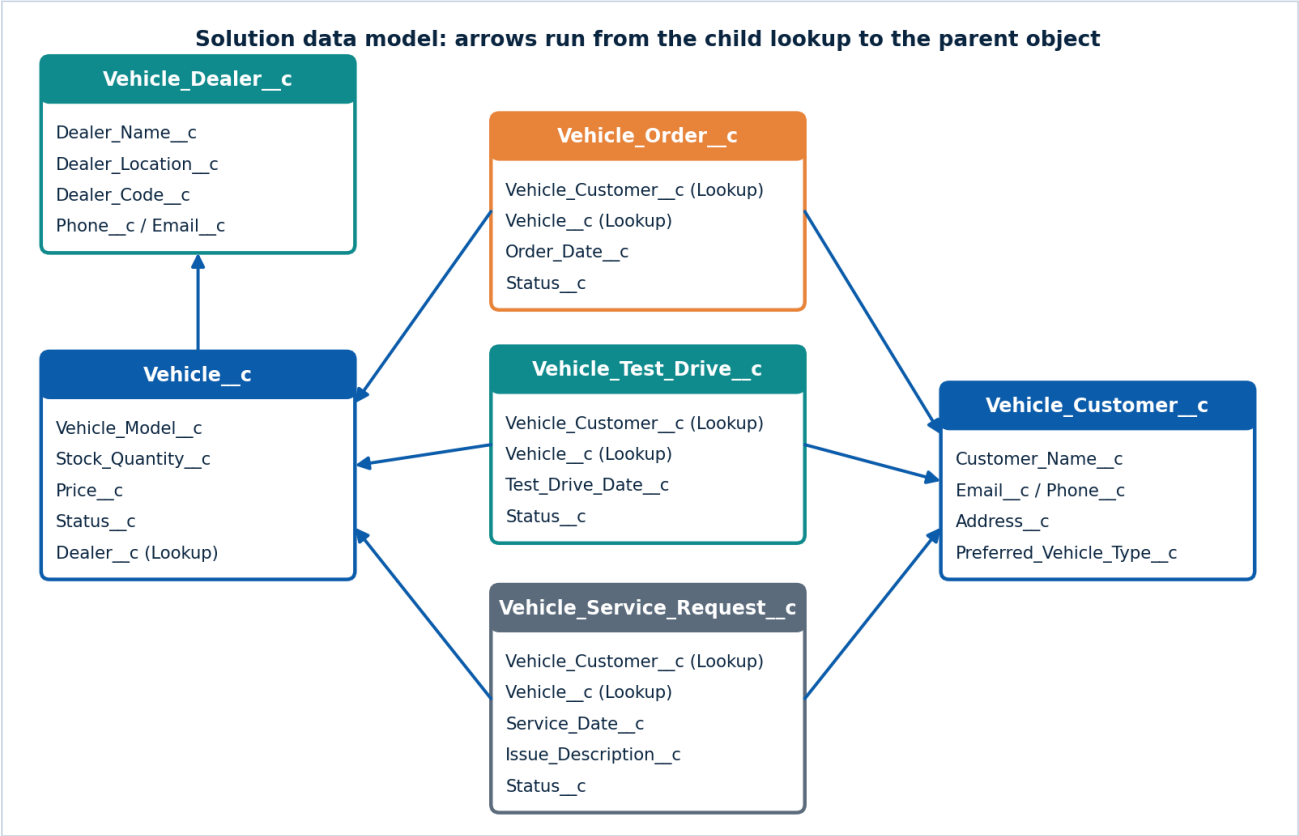


Figure 4: Data model of the solution and the lookup relationships between objects

Layer	Components
Presentation	WhatNext Vision Motors Lightning app, six custom tabs, Reports and Dashboards tabs
Data	Vehicle__c, Vehicle_Dealer__c, Vehicle_Customer__c, Vehicle_Order__c, Vehicle_Test_Drive__c, Vehicle_Service_Request__c
Automation	Auto Assign Dealer flow, Test Drive Reminder flow
Apex	VehicleOrderTrigger, VehicleOrderTriggerHandler, VehicleOrderBatch, VehicleOrderBatchScheduler

5. Project Planning and Scheduling

5.1 Project Planning

- The work follows an **Agile** approach with six one-week sprints.
- Requirements are written as epics, split into user stories, and each story is given story points for effort.
- Velocity (points completed per sprint) is used to judge whether the remaining sprints are realistic.
- Sprints 1-4 cover setup, data model, automation and Apex. Sprints 5-6 cover reporting and are planned.

5.2 Product Backlog and Sprint Schedule

Sprint	Epic	ID	User story	Pts	Priority	Owner
1	Developer setup	USN-1	As an administrator, I want a Salesforce Developer org so that I can build and test the order management system.	3	High	Member 1
2	Data modelling	USN-2	As an administrator, I want Vehicle, Dealer, Customer, Order, Test Drive and Service Request objects with relationships so that business data is stored in an organised way.	5	High	Member 2
2	Data modelling	USN-3	As a staff member, I want tabs and a Lightning app for every module so that I can find and manage records quickly.	5	High	Member 3
3	Automation	USN-4	As the business, I want orders for out-of-stock vehicles to be rejected so that customers are not misled.	3	High	Member 4
3	Automation	USN-5	As a sales coordinator, I want a flow to assign the dealer for a new order so that manual allocation is not needed.	3	High	Member 4
4	Apex	USN-6	As the business, I want an Apex trigger and handler to enforce stock rules instantly and in bulk.	5	High	Member 1
4	Apex	USN-7	As an inventory manager, I want a scheduled batch job to process pending orders against current stock so that statuses stay accurate.	5	High	Member 2
5	Reports	USN-8	As a manager, I want reports on order status and dealer activity so that I can monitor operations.	4	High	Member 3
6	Dashboards	USN-9	As a manager, I want a dashboard that summarises orders, dealers and sales so that I can decide quickly.	4	Medium	Member 4

Owner names follow the team-member slots of the project template and can be replaced with the actual team members.

5.3 Sprint Tracker, Velocity and Burndown

Sprint	Story points	Duration	Start date	Planned end	Points completed	Release date (actual)
Sprint 1	3	6 days	07 Sep 2026	12 Sep 2026	3	12 Sep 2026
Sprint 2	10	6 days	14 Sep 2026	19 Sep 2026	10	19 Sep 2026
Sprint 3	6	6 days	21 Sep 2026	26 Sep 2026	6	26 Sep 2026
Sprint 4	10	6 days	28 Sep 2026	03 Oct 2026	10	03 Oct 2026
Sprint 5	4	6 days	05 Oct 2026	10 Oct 2026	Planned	Planned
Sprint 6	4	6 days	12 Oct 2026	17 Oct 2026	Planned	Planned

Velocity. Sprints 1-4 delivered 29 of the 37 story points, an average velocity of 7.25 points per sprint (29 points over 4 sprints, about 1.2 points per day). The remaining 8 points need roughly one sprint of work at this pace, so the two planned sprints leave a comfortable margin.

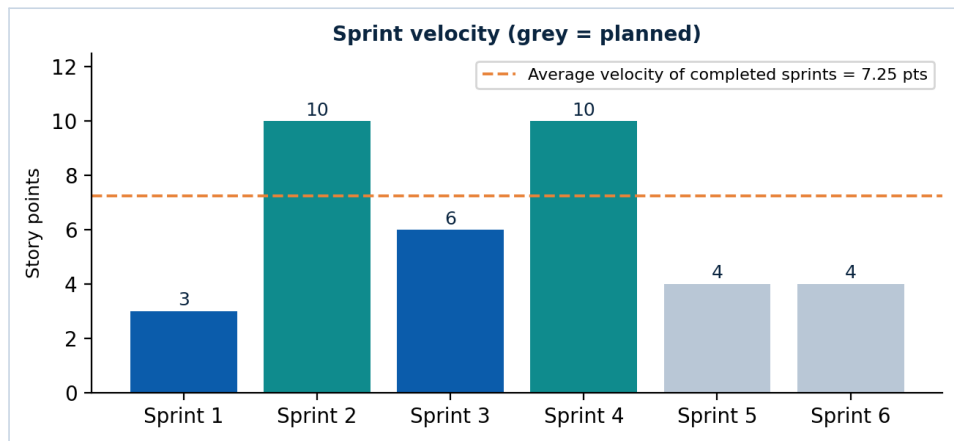


Figure 5: Story points per sprint

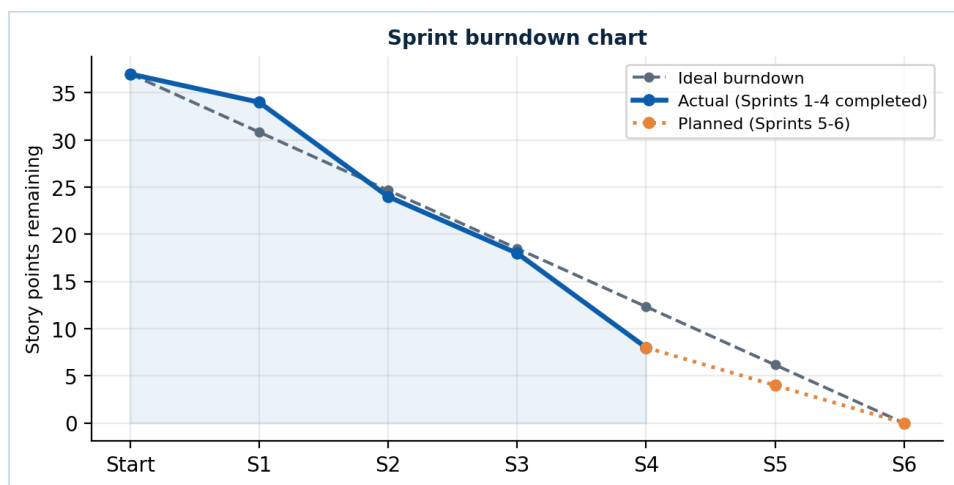


Figure 6: Burndown of story points across the six sprints

Sprint dates are illustrative planning dates and should be adjusted to match the actual working calendar.

6. Project Development Phase

Development was carried out in seven milestones. Each milestone below states what was built, the steps followed, and the evidence from the Salesforce org.

Milestone	Topic	Outcome
1	Salesforce account	Developer Edition org ready
2	Data management: objects	Six custom objects deployed
3	Data management: tabs	Tab for every object
4	Data management: app manager	WhatNext Vision Motors Lightning app
5	Data management: fields	Fields and lookup relationships
6	Automation	Two active record-triggered flows
7	Apex and batch class	Trigger, batch job and daily schedule

Milestone 1: Salesforce Account

A free Salesforce Developer Edition org provides a full copy of the platform for building and testing.

- Open the Salesforce Developer sign-up page and fill in first name, last name, email, role (Developer), company (college name), country (India), postal code and a unique username in email format.
- Open the verification email and select **Verify Account**. The mail can take 5–10 minutes to arrive.
- Set a password, choose a security question and answer, and select **Change Password**.
- Salesforce then opens the Setup home page, which is the starting point for every later milestone.

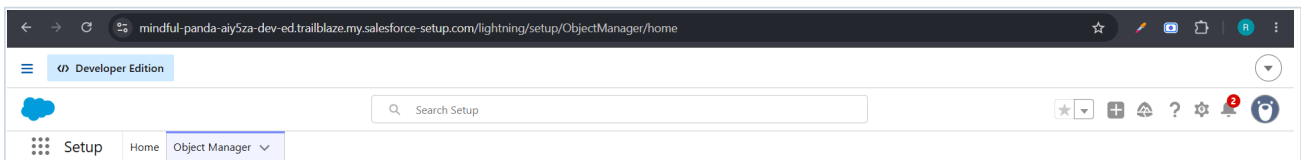


Figure 7: Developer Edition org with the Setup home and Object Manager available

Milestone 2: Data Management - Objects

Custom objects hold the business data. Six objects were created and related to one another.

Object	Purpose	Relationships
Vehicle__c	Stores vehicle details and stock	Related to Dealer and Orders
Vehicle_Dealer__c	Stores authorised dealer information	Related to Orders
Vehicle_Customer__c	Stores customer details	Related to Orders and Test Drives
Vehicle_Order__c	Tracks vehicle purchases	Related to Customer and Vehicle
Vehicle_Test_Drive__c	Tracks test drive bookings	Related to Customer and Vehicle
Vehicle_Service_Request__c	Tracks servicing requests	Related to Customer and Vehicle

Steps to create an object

- Gear icon → **Setup** → **Object Manager** → **Create** → **Custom Object**.

- Enter the label (for example *Vehicle*) and plural label (*Vehicles*), then set the record name label (*Vehicle Name*) with data type **Text**.
- Tick **Allow Reports** and **Allow Search**, keep the deployment status as **Deployed**, and select **Save**.
- Repeat for Vehicle Dealer (record name *Dealer Name*) and the four remaining objects.

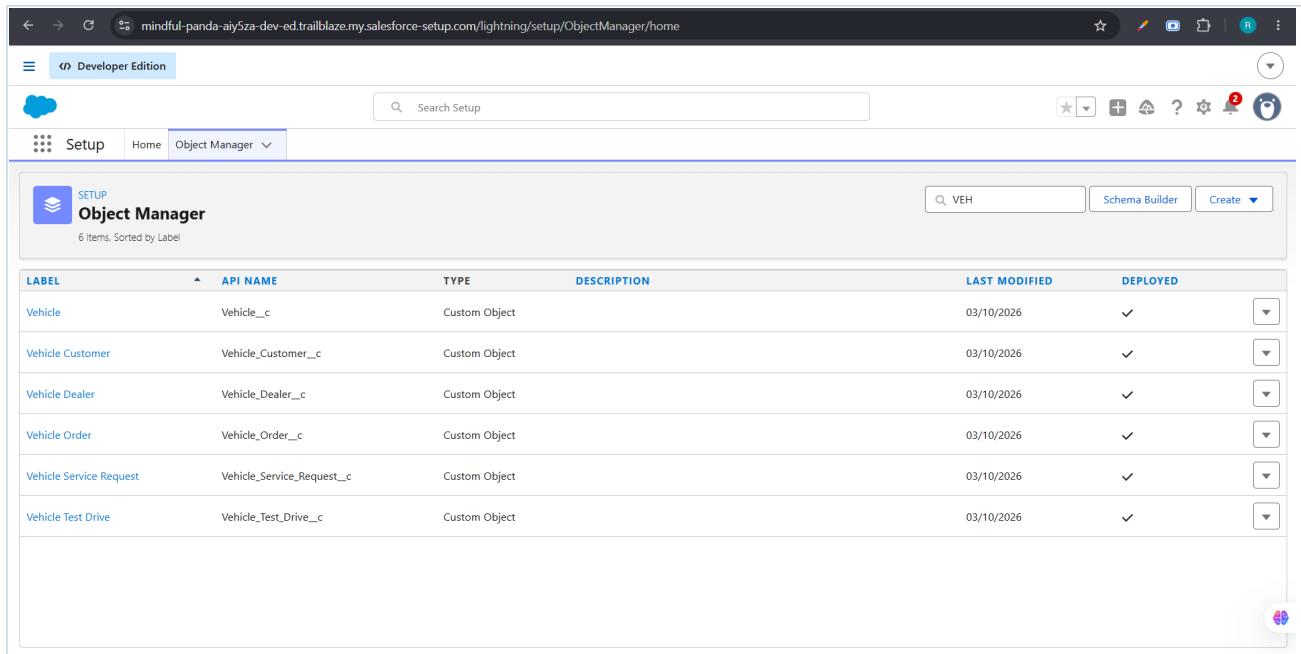


Figure 8: Object Manager filtered by "VEH", showing the six custom objects, all deployed

Milestone 3: Data Management - Tabs

Tabs let users open an object's records directly from the app navigation.

- In Setup, search for **Tabs** in Quick Find and open the page.
- Under **Custom Object Tabs**, select **New**, pick the object (for example Vehicle) and choose a tab style.
- Accept the defaults on the "Add to profiles" and "Add to custom apps" pages and select **Save**.
- Repeat for the other five objects. Each tab was given a different style so that it is easy to recognise.

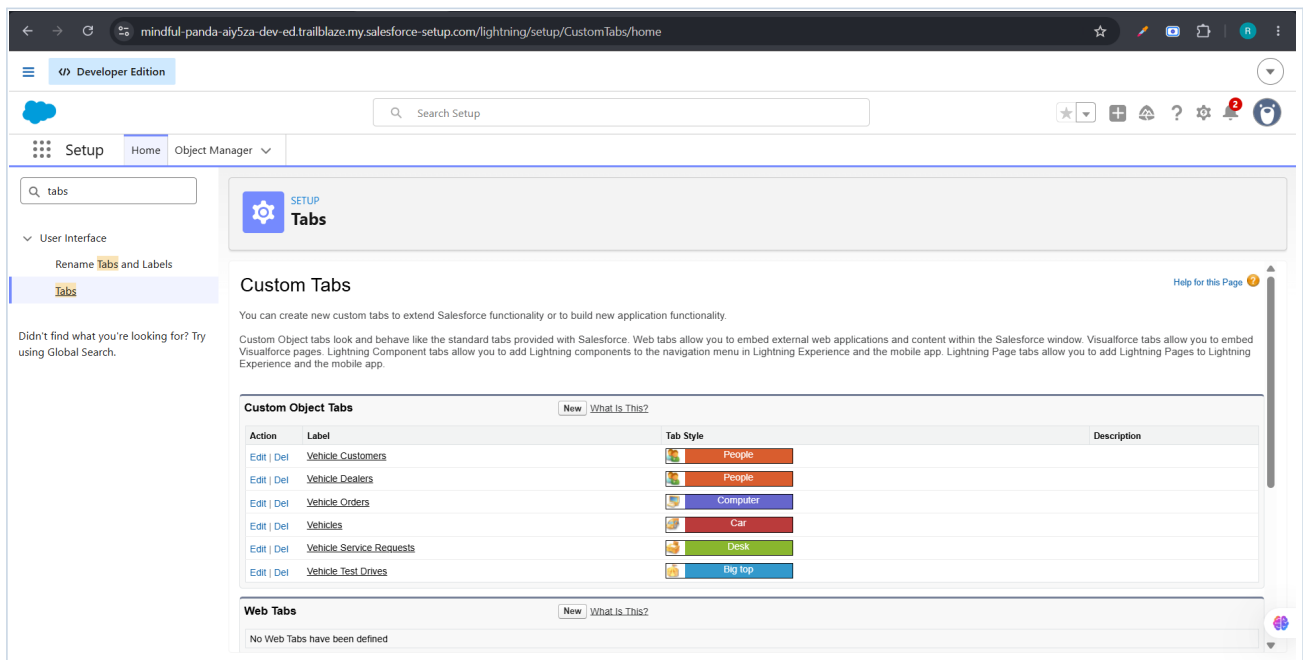


Figure 9: Custom Object Tabs created for all six objects, each with its own tab style

Milestone 4: Data Management - App Manager

A Lightning app groups the tabs a team needs into one workspace.

- In Setup, search for **App Manager** and select **New Lightning App**.
- Enter the app name **WhatNext Vision Motors**; the developer name fills in automatically. Add a short description and keep the default colour.
- Keep the default app options and utility items, then add navigation items: Vehicle Customers, Vehicle Dealers, Vehicle Orders, Vehicle Service Requests, Vehicle Test Drives, Vehicles, Reports and Dashboards.
- Assign the **System Administrator** profile and select **Save & Finish**.

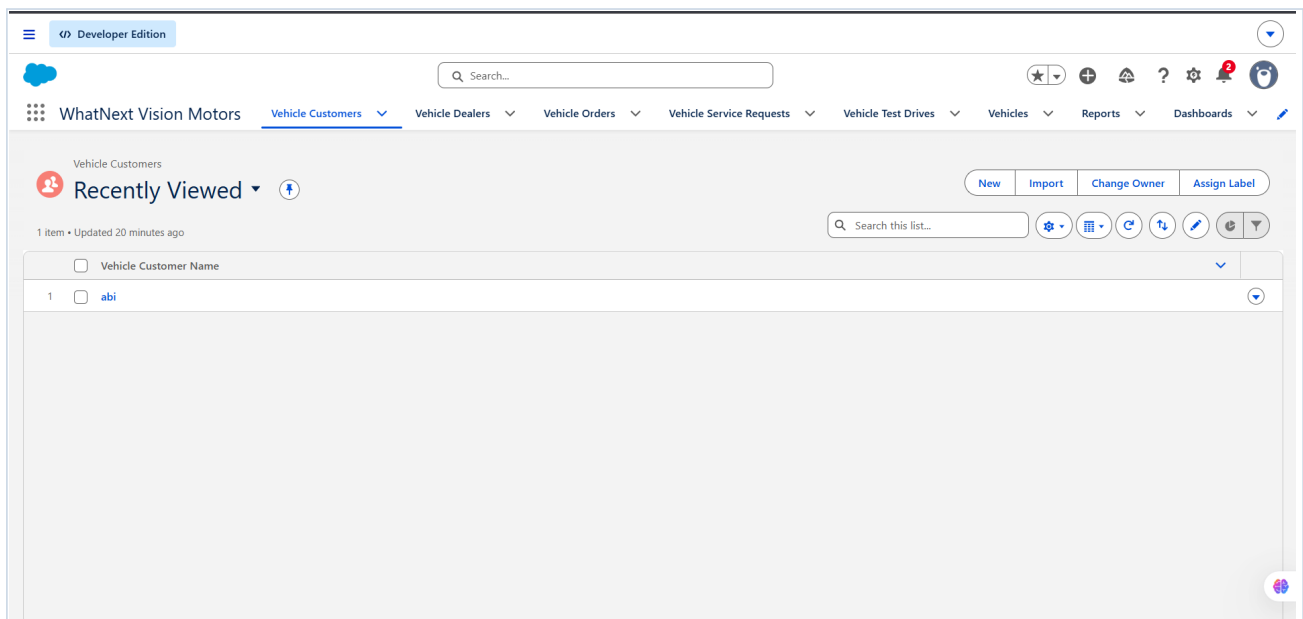


Figure 10: WhatNext Vision Motors Lightning app with all navigation items; the Vehicle Customers list shows a saved customer record

Milestone 5: Data Management - Fields

Fields describe each record. The table lists the key fields; the record name of each object is the standard *Name* field.

Object	Fields (data type)
Vehicle__c	Vehicle Name (Text, record name); Vehicle_Model__c (Picklist: Sedan, SUV, EV, etc.); Stock_Quantity__c (Number); Price__c (Currency); Dealer__c (Lookup to Vehicle Dealer); Status__c (Picklist: Available, Out of Stock, Discontinued)
Vehicle_Dealer__c	Dealer Name (Text, record name); Dealer_Location__c (Text); Dealer_Code__c (Auto Number); Phone__c (Phone); Email__c (Email)
Vehicle_Customer__c	Customer Name (Text, record name); Email__c (Email); Phone__c (Phone); Address__c (Text); Preferred_Vehicle_Type__c (Picklist: Sedan, SUV, EV, etc.)
Vehicle_Order__c	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Order_Date__c (Date); Status__c (Picklist: Pending, Confirmed, Delivered, Canceled)
Vehicle_Test_Drive__c	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Test_Drive_Date__c (Date); Status__c (Picklist: Scheduled, Completed, Canceled)
Vehicle_Service_Request__c	Vehicle_Customer__c (Lookup); Vehicle__c (Lookup); Service_Date__c (Date); Issue_Description__c (Text); Status__c (Picklist: Requested, In Progress, Completed)

Steps to create a field

- Setup → **Object Manager** → open the object → **Fields & Relationships** → **New**.
- **Picklist** (for example Vehicle Model): choose the type, enter the label, type one value per line, then **Next** → **Next** → **Save & New**.
- **Number** (Stock Quantity): choose the type, enter the label and length, then continue to save.
- **Lookup relationship** (Dealer on Vehicle): choose *Lookup Relationship*, select the related object (Vehicle Dealer), name the field and keep the default field-level security.

The screenshot shows the Salesforce Object Manager interface for the 'Vehicle' object. The 'Fields & Relationships' section is active, displaying a table of 9 fields. The table has columns for Field Label, Field Name, Data Type, Controlling Field, and Indexed status. The fields listed are: Created By (Lookup(User)), Dealer (Lookup(Vehicle Dealer)), Last Modified By (Lookup(User)), Owner (Lookup(User, Group)), Price (Currency(18, 0)), Status (Picklist), Stock Quantity (Number(4, 0)), and Vehicle Model (Picklist). The Dealer, Owner, and Stock Quantity fields are marked as indexed.

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedById	Lookup(User)		
Dealer	Dealer__c	Lookup(Vehicle Dealer)		✓
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User, Group)		✓
Price	Price__c	Currency(18, 0)		
Status	Status__c	Picklist		
Stock Quantity	Stock_Quantity__c	Number(4, 0)		
Vehicle Model	Vehicle_Model__c	Picklist		

Figure 11: Fields and Relationships of the Vehicle object: Dealer lookup, Price, Status, Stock Quantity and Vehicle Model

Milestone 6: Automation

Two record-triggered flows automate the order and test-drive processes without any code.

Flow 1: Auto Assign Dealer

Goal: when a Vehicle Order is created, find the dealer that serves the customer's location. Entry conditions make the flow run only for new orders that reference a customer, because the next step needs the customer's address.

Step	Element	Configuration
1	Start: Record-Triggered Flow	Object: Vehicle Order. Trigger: a record is created. Runs immediately.
2	Get Records: <i>Get Customer Information</i>	Object: Vehicle Customer. Condition: Id equals {!\$Record.Vehicle_Customer__c}.
3	Get Records: <i>Get Nearest Dealer</i>	Object: Vehicle Dealer. Condition: Dealer_Location__c equals {!Get_Customer_Information.Address__c}.
4	Update Records: <i>Assign Dealer to Order</i>	Uses the IDs and all field values from a record: Vehicle Dealer from Get Nearest Dealer.
5	End	The flow was saved as Auto Assign Dealer and activated.

The screenshot displays the Salesforce Flow Builder interface for the 'Auto Assign Dealer (V1)' flow. The flow is active, as indicated by the 'Active' status in the top right. The flow diagram on the left shows a sequence of steps: 'Run Immediately' (Start), 'Get Customer Information' (Get Records), 'Get Nearest Dealer' (Get Records), 'Assign Dealer to Order' (Update Records), and 'End'. The 'Assign Dealer to Order' step is currently selected, and its configuration pane is open on the right. This pane shows the label 'Assign Dealer to Order', the API Name 'Assign_Dealer_to_Order', and the record collection 'Vehicle Dealer from Get Nearest Dealer'. The 'How to Find Records to Update and Set Their Values' section is also visible, with the option 'Use the IDs and all field values from a record or record collection' selected.

Figure 12: Auto Assign Dealer (V1) in Flow Builder: active, with the Assign Dealer to Order element selected

Flow 2: Test Drive Reminder

Goal: email the customer one day before a scheduled test drive. The flow uses a **scheduled path**, which waits until a date or time value on the record before running.

Step	Element	Configuration
1	Start: Record-Triggered Flow	Object: Vehicle Test Drive. Trigger: a record is created or updated. Entry condition: Status equals Scheduled.
2	Scheduled path: <i>Reminder Before Test Drive</i>	Time source: Test_Drive_Date__c. Offset: 1 day before. (The flow therefore has two paths: Run Immediately and the reminder path.)
3	Get Records: <i>Get Customer Information</i>	Object: Vehicle Customer. Condition: Id equals {!\$Record.Vehicle_Customer__c}. Store the first record with all fields.
4	Action: <i>Send Test Drive Reminder</i>	Core action Send Email. Recipient: the customer's Email from step 3. Subject: "Reminder: Your Test Drive is Tomorrow!". Rich-text body: true.
5	Text template: <i>EmailSent</i>	Body that greets the customer by name, mentions the vehicle and the date (tomorrow), and tells them how to reschedule.

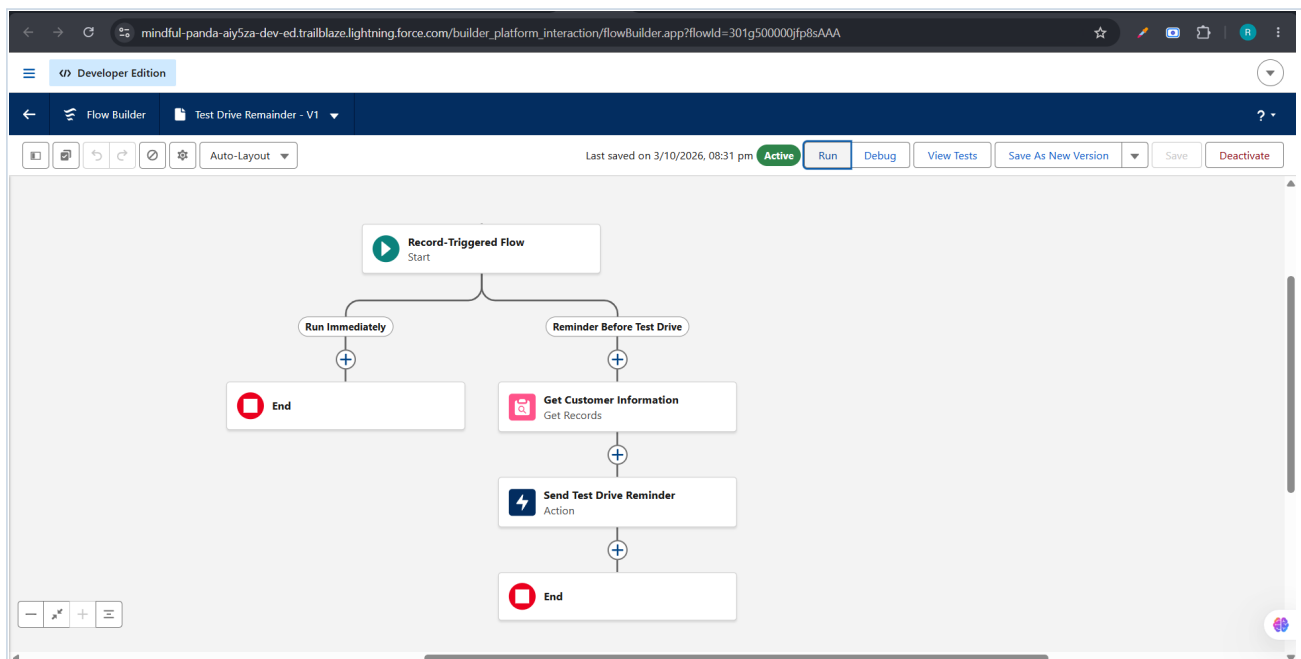


Figure 13: Test Drive Reminder flow: Run Immediately path ends at once; the reminder path gets the customer and sends the email

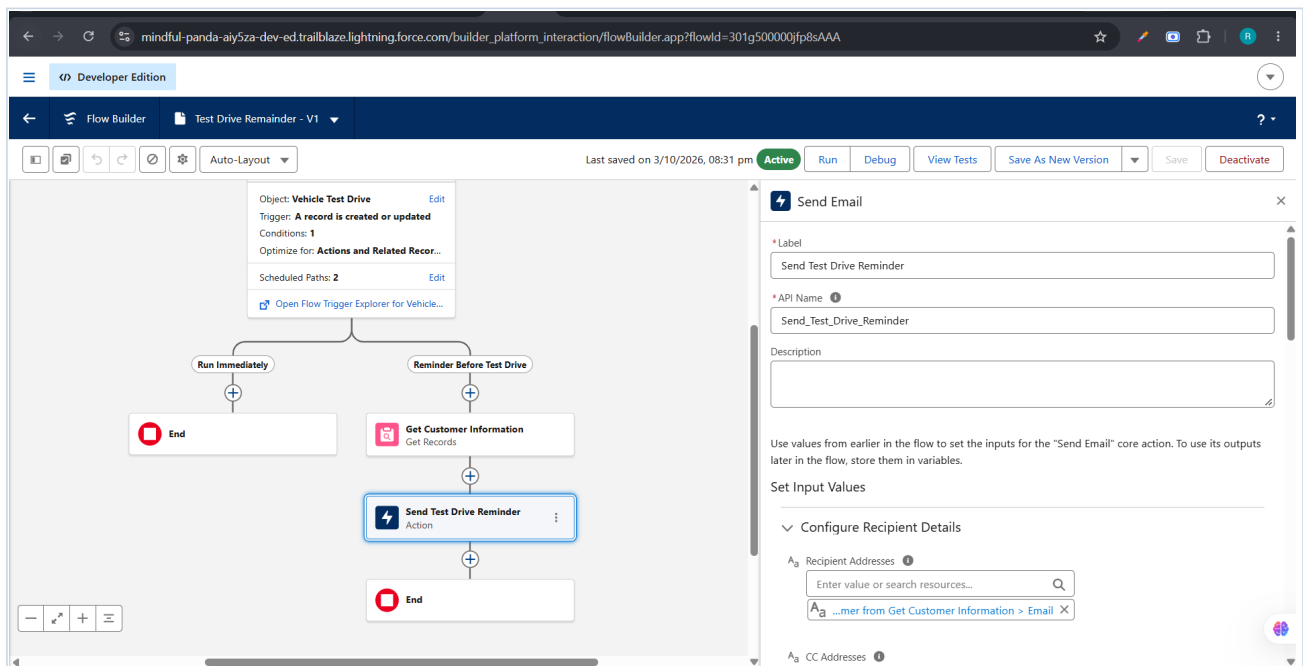


Figure 14: Send Email action using the customer's email address as recipient (flow shown as Active)

In the org this flow carries the label *Test Drive Remainder*; the name can be corrected by saving a new version with the label "Test Drive Reminder".

Milestone 7: Apex and Batch Class

Some rules are better enforced in code. Apex runs on the server, handles many records at once, and cannot be bypassed from the user interface.

Component	Type	Role
VehicleOrderTrigger	Apex trigger	Fires before and after insert and update of Vehicle Order and delegates to the handler
VehicleOrderTriggerHandler	Apex class	Rejects orders for out-of-stock vehicles (before) and reduces stock for confirmed orders (after)
VehicleOrderBatch	Batch Apex	Reads all orders in Pending status and confirms those whose vehicle has stock
VehicleOrderBatchScheduler	Schedulable Apex	Starts the batch job with a batch size of 50

Steps

- Open the **Developer Console** from the gear icon.
- **File → New → Apex Class**, name it *VehicleOrderTriggerHandler* and add the code below.
- **File → New → Apex Trigger**, name it *VehicleOrderTrigger*, choose the *Vehicle_Order__c* object and call the handler.
- Create *VehicleOrderBatch* and *VehicleOrderBatchScheduler* as Apex classes.
- Use **Debug → Open Execute Anonymous Window** to schedule the job.

Trigger handler class

```
public class VehicleOrderTriggerHandler {

    public static void handleTrigger(List<Vehicle_Order__c> newOrders, Map<Id, Vehicle_Order__c> oldOrders,
        Boolean isBefore, Boolean isAfter, Boolean isInsert, Boolean isUpdate) {
        if (isBefore && (isInsert || isUpdate)) {
            preventOrderIfOutOfStock(newOrders);
        }
        if (isAfter && (isInsert || isUpdate)) {
            updateStockOnOrderPlacement(newOrders);
        }
    }

    // Reject the order when the selected vehicle has no stock left
    private static void preventOrderIfOutOfStock(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (vehicleIds.isEmpty()) { return; }

        Map<Id, Vehicle__c> stockMap = new Map<Id, Vehicle__c>(
            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]);

        for (Vehicle_Order__c order : orders) {
            Vehicle__c vehicle = stockMap.get(order.Vehicle__c);
            if (vehicle != null && vehicle.Stock_Quantity__c <= 0) {
                order.addError('This vehicle is out of stock. Order cannot be placed.');
            }
        }
    }

    // Reduce stock by one for each confirmed order

    private static void updateStockOnOrderPlacement(List<Vehicle_Order__c> orders) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orders) {
            if (order.Vehicle__c != null && order.Status__c == 'Confirmed') {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (vehicleIds.isEmpty()) { return; }

        Map<Id, Vehicle__c> stockMap = new Map<Id, Vehicle__c>(
            [SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]);

        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();
        for (Vehicle_Order__c order : orders) {
            Vehicle__c vehicle = stockMap.get(order.Vehicle__c);
            if (vehicle != null && vehicle.Stock_Quantity__c > 0) {
                vehicle.Stock_Quantity__c -= 1;
                vehiclesToUpdate.add(vehicle);
            }
        }
        if (!vehiclesToUpdate.isEmpty()) {
            update vehiclesToUpdate;
        }
    }
}
```

Trigger

```
trigger VehicleOrderTrigger on Vehicle_Order__c (before insert, before update, after insert, after update) {
    VehicleOrderTriggerHandler.handleTrigger(trigger.new, trigger.oldMap,
        trigger.isBefore, trigger.isAfter, trigger.isInsert, trigger.isUpdate);
}
```

Batch class: process pending orders

```
global class VehicleOrderBatch implements Database.Batchable<sObject> {

    global Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Status__c, Vehicle__c
            FROM Vehicle_Order__c
            WHERE Status__c = 'Pending'
        ]);
    }

    global void execute(Database.BatchableContext bc, List<Vehicle_Order__c> orderList) {
        Set<Id> vehicleIds = new Set<Id>();
        for (Vehicle_Order__c order : orderList) {
            if (order.Vehicle__c != null) {
                vehicleIds.add(order.Vehicle__c);
            }
        }
        if (vehicleIds.isEmpty()) { return; }

        Map<Id, Vehicle__c> stockMap = new Map<Id, Vehicle__c>([
            SELECT Id, Stock_Quantity__c FROM Vehicle__c WHERE Id IN :vehicleIds]);

        List<Vehicle_Order__c> ordersToUpdate = new List<Vehicle_Order__c>();
        List<Vehicle__c> vehiclesToUpdate = new List<Vehicle__c>();

        for (Vehicle_Order__c order : orderList) {
            Vehicle__c vehicle = stockMap.get(order.Vehicle__c);
            if (vehicle != null && vehicle.Stock_Quantity__c > 0) {
                order.Status__c = 'Confirmed';
                vehicle.Stock_Quantity__c -= 1;
                ordersToUpdate.add(order);
                vehiclesToUpdate.add(vehicle);
            }
        }

        if (!ordersToUpdate.isEmpty()) { update ordersToUpdate; }
        if (!vehiclesToUpdate.isEmpty()) { update vehiclesToUpdate; }
    }

    global void finish(Database.BatchableContext bc) {
        System.debug('Vehicle order batch job completed.');
```

Schedulable class

```
global class VehicleOrderBatchScheduler implements Schedulable {
    global void execute(SchedulableContext sc) {
        VehicleOrderBatch batchJob = new VehicleOrderBatch();
        Database.executeBatch(batchJob, 50); // 50 records per batch
    }
}
```

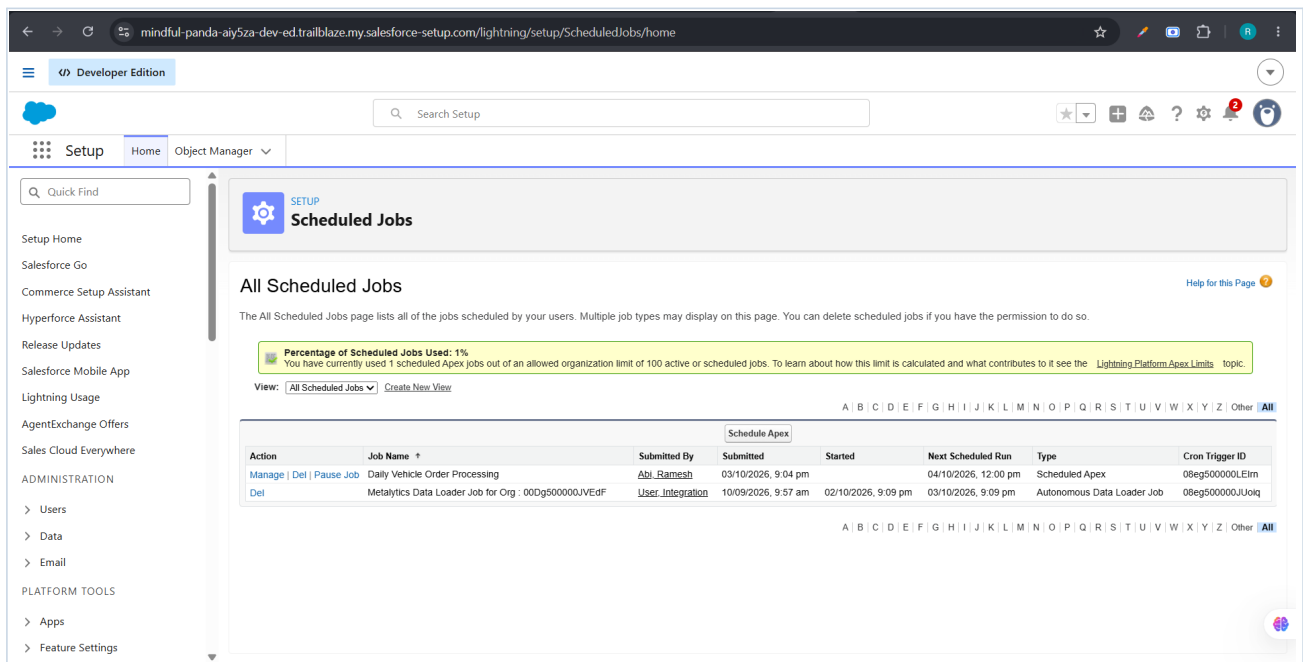
Scheduling the job

The job is registered once from the Execute Anonymous window of the Developer Console. The cron expression below runs it every day at 12:00 PM.

```
String cronExp = '0 0 12 * * ?'; // seconds minutes hours day-of-month month day-of-week
System.schedule('Daily Vehicle Order Processing', cronExp, new VehicleOrderBatchScheduler());
```

Seconds	Minutes	Hours	Day of month	Month	Day of week
0	0	12	* (every day)	* (every month)	? (no specific day)

After scheduling, the job appears under **Setup → Scheduled Jobs** with its next run time.



The screenshot displays the Salesforce Setup page for Scheduled Jobs. The page title is "Scheduled Jobs" and it includes a search bar and a "Search Setup" button. The left sidebar shows the Setup menu with options like Setup Home, Salesforce Go, Commerce Setup Assistant, Hyperforce Assistant, Release Updates, Salesforce Mobile App, Lightning Usage, AgentExchange Offers, Sales Cloud Everywhere, ADMINISTRATION, and PLATFORM TOOLS. The main content area shows "All Scheduled Jobs" with a description: "The All Scheduled Jobs page lists all of the jobs scheduled by your users. Multiple job types may display on this page. You can delete scheduled jobs if you have the permission to do so." A yellow banner indicates "Percentage of Scheduled Jobs Used: 1%" and provides a link to "Lightning Platform Apex Limits". Below this, there is a table of scheduled jobs. The table has columns for Action, Job Name, Submitted By, Submitted, Started, Next Scheduled Run, Type, and Cron Trigger ID. The first job listed is "Daily Vehicle Order Processing" submitted by "Abi_Ramesh" on 03/10/2026 at 9:04 pm, with a next scheduled run on 04/10/2026 at 12:00 pm. The second job is "Metalytics Data Loader Job for Org : 00Dg500000JVEdf" submitted by "User_Integration" on 10/09/2026 at 9:57 am, with a next scheduled run on 02/10/2026 at 9:09 pm.

Action	Job Name	Submitted By	Submitted	Started	Next Scheduled Run	Type	Cron Trigger ID
Manage Del Pause Job	Daily Vehicle Order Processing	Abi_Ramesh	03/10/2026, 9:04 pm		04/10/2026, 12:00 pm	Scheduled Apex	08eg500000LElm
Del	Metalytics Data Loader Job for Org : 00Dg500000JVEdf	User_Integration	10/09/2026, 9:57 am	02/10/2026, 9:09 pm	03/10/2026, 9:09 pm	Autonomous Data Loader Job	08eg500000JUoik

Figure 15: Scheduled Jobs page: Daily Vehicle Order Processing (Scheduled Apex) with its next run on 04/10/2026 at 12:00 pm

7. Functional and Performance Testing

7.1 Functional test cases

Each feature was checked directly in the org. Where a screenshot supports the result, the figure is referenced; the two remaining cases need a screenshot to be captured when the scenario is run.

ID	Scenario	Expected result	Status
TC-01	Check the custom objects in Object Manager	Six Vehicle objects exist and are deployed	Passed (Figure 8)
TC-02	Open the custom tabs setup page	A tab exists for every object	Passed (Figure 9)
TC-03	Open the WhatNext Vision Motors app and the Vehicle Customers tab	All navigation items appear and the customer record is listed	Passed (Figure 10)
TC-04	Review the Vehicle fields	Dealer lookup, Price, Status, Stock Quantity and Vehicle Model fields exist	Passed (Figure 11)
TC-05	Open the Vehicle Test Drives list	A test drive record (TD-001) is saved and listed	Passed (Figure 16)
TC-06	Open Flow Builder for both flows	Auto Assign Dealer and Test Drive Reminder show status Active	Passed (Figures 12 and 13)
TC-07	Open Scheduled Jobs	Daily Vehicle Order Processing is registered with a next run time	Passed (Figure 15)
TC-08	Create an order for a vehicle with stock quantity 0	Save is blocked with the message "This vehicle is out of stock. Order cannot be placed."	Screenshot to be added
TC-09	Save a test drive with Status = Scheduled for tomorrow	Customer receives the reminder email the day before	Screenshot to be added

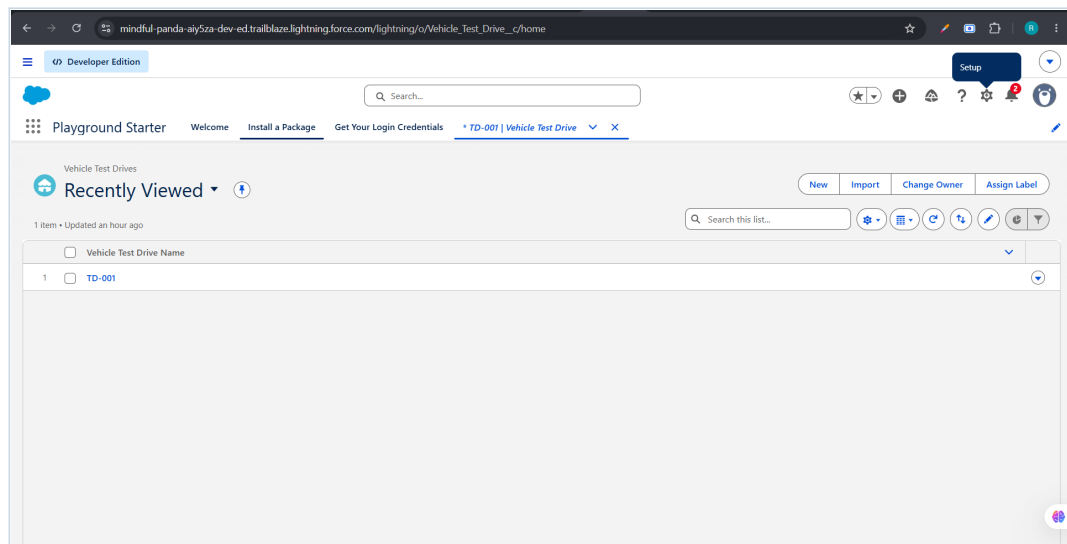


Figure 16: Vehicle Test Drives list showing the saved test drive record TD-001

7.2 Performance observations

Aspect	Observation
Scheduled jobs limit	The Scheduled Jobs page reports 1% of the allowed 100 active or scheduled jobs in use (1 scheduled Apex job)
Batch size	The batch job processes 50 orders per execution, well within governor limits
Trigger design	Handler collects IDs first and runs one query for all orders in the transaction, so it is bulk-safe
Flow design	Entry conditions and a single lookup per element keep each flow interview short

8. Advantages and Disadvantages

Advantages

- **Centralised data:** vehicles, dealers, customers and orders live in one connected system.
- **Automated workflows:** dealer lookup, reminders and status updates run without staff effort.
- **Real-time visibility:** reports and dashboards read the same live records.
- **Reliable rules:** stock validation is enforced on the server for every order.
- **Easy to grow:** new dealers, vehicles and locations need only new records.

Disadvantages

- The solution depends on the Salesforce platform and its licensing and limits.
- Initial configuration (objects, flows, Apex) takes time and Salesforce skills.

Known limitations and recommended improvements

- **Dealer matching is by exact text.** The flow compares the dealer location with the customer address, so it finds a dealer in the same named location rather than the geographically nearest one. A Geolocation field with a distance calculation would make “nearest” literal.
- **Stock reduction should run once per order.** The after-save method and the batch job both reduce stock and can run again when a confirmed order is edited. Reducing stock only when the status changes to Confirmed (comparing with the old value) would prevent double counting.
- **Stock replenishment emails** mentioned in the use case are not yet part of the batch job.

9. Conclusion

The WhatNext Vision Motors CRM shows how Salesforce can move a vehicle business from scattered, manual order handling to a connected and automated process. Six related objects hold the data, a Lightning app gives users a single workspace, flows assign dealers and send test-drive reminders, and Apex protects the integrity of stock while a scheduled batch job keeps pending orders moving. The result is fewer order errors, quicker responses and clearer communication with customers. The project also gave practical experience of data modelling, Lightning App Builder, record-triggered flows, Apex triggers, Batch Apex and Scheduled Apex.

10. Future Scope

- **Mobile ordering app** so customers can place and track orders from a phone.
- **GPS-based dealer lookup** using Geolocation fields to pick the truly nearest dealer.
- **SMS and WhatsApp notifications** alongside email for reminders and status changes.
- **AI demand forecasting** to predict which models need restocking.

- **Stock replenishment alerts** from the batch job to the inventory team.
- **Ready-made reports and dashboards** for order status, dealer performance and sales.

11. Appendix

A. Project artefacts

Artefact	Location / description
Source code	Apex classes and trigger in the Developer Console of the Salesforce Developer org
Dataset	Salesforce custom objects: Vehicle, Vehicle Dealer, Vehicle Customer, Vehicle Order, Vehicle Test Drive, Vehicle Service Request
Automation	Flows: Auto Assign Dealer, Test Drive Reminder
Demo	WhatNext Vision Motors Lightning app in the Developer Edition org

B. Glossary

Term	Meaning
Custom object	A database table created by the user to store business data
Lookup relationship	A link from a child record to a parent record without strict ownership
Record-triggered flow	An automation that runs when a record is created, updated or deleted
Scheduled path	A flow branch that runs at a set time relative to a date field on the record
Apex trigger	Code that runs before or after records are saved
Batch Apex	Code that processes large numbers of records in smaller chunks
Schedulable Apex	Code that can be set to run automatically at fixed times using a cron expression

12. Team Details and Project Links

A. Academic details

Item	Details
Degree	B.E. Computer Science and Engineering
College name	A.V.C. College of Engineering

B. Project team

Role	Name	Email ID
Team lead	R. Gayathri	gayathrijaga09@gmail.com
Member	R. Abinaya	abiramesh524@gmail.com
Member	N. Karthika	karthikank0602@gmail.com
Member	R. Kavidharshini	kavidharshini0916@gmail.com
Member	P. Harini	hariniprithivirajan@gmail.com

C. Project links

Item	Link
Project demo (Google Drive)	https://drive.google.com/file/d/1oFjijb51HnnY6tKxr-HRSrbsJguMI7Y6/view?usp=drivesdk
GitHub repository	https://github.com/RAbinaya2122/WhatNext-Vision-Motors.git

Thank You